

Examen réparti 2
Master 1 SAR
UE 4I401

L. Arantes, P. Cadinot, S. Dubois, J. Lejeune, J. Sopena, P. Sens

Janvier 2019

2 heures – **Tout document papier autorisé**

Tout appareil électronique interdit

Le barème sur 20 points est donné à titre indicatif

Exercice 1 – Questions de cours (1,5 points)

Question 1 $\frac{1}{2}$ point

Lors du traitement d'une interruption disque, est-il possible de traiter une interruption horloge ? Justifiez votre réponse.

Question 2 $\frac{1}{2}$ point

Lorsqu'un signal est envoyé à un processus, quand, au plus tôt, ce processus traitera le signal et dans quel mode (utilisateur ou système) ?

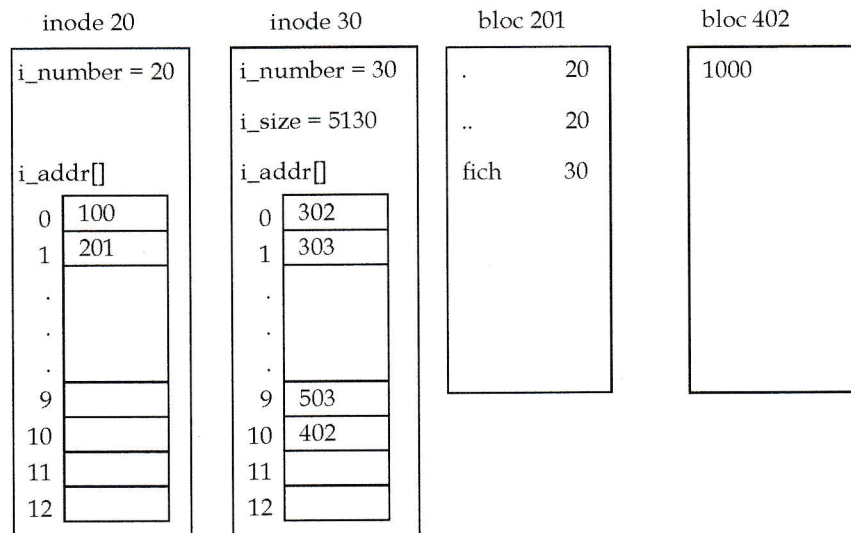
Question 3 $\frac{1}{2}$ point

Est-il possible qu'un processus soit à l'état prêt (SRUN) et reste non chargé en mémoire ? Si oui dans quel cas. Justifiez vos réponses.

Exercice 2 – Système de fichiers et Buffer Cache (6,5 points)

Les *inodes* ont une structure similaire à celle vue en TD et rappelée en annexe. La table des inodes sur disque commence au bloc 2. Les inodes ont une taille de 32 octets et leur numéro commence à partir de 1. Les blocs ont une taille de 512 octets.

Le répertoire racine correspond à l'inode numéro 20. Nous considérons les *inodes* et les blocs suivants :



Le bloc 100 ne contient pas la chaîne de caractère "fich".

Question 1 1/2 point

Quel est le numéro du dernier bloc de données du fichier "fich" ?

Question 2 1/2 point

Dans quels blocs sont stockés les inodes 20 et 30 ? Justifiez votre réponse.

Soit le code utilisateur suivant :

```

1  int f1, f2;
2  f1 = open("/fich", O_WRONLY | O_APPEND); /* O_APPEND
    positionne en fin de fichier */
3  f2 = open("/fich", O_RDWR);
4  if (fork() == 0) {
5      write(f1, "aaa", 3);
6      if (fork() == 0) {
7          close(f2);
8          write(f1, "bbb", 3);
9          exit(0);
10     }
11     wait(NULL);
12     exit(0);
13 }
14 wait(NULL);
15 close(f1); close(f2);
16 exit(0);
17 }

```

Question 3 1 1/2 points

Donnez l'état, juste après la ligne 8, des tables de descripteurs de fichiers, fichiers ouverts, et inodes (faites un schéma, pensez à représenter notamment les champs *f_count*, *f_offset*, *f_flags*, *i_count*, *i_size* et *i_number*).

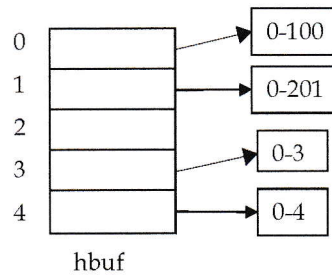
Question 4 2 points

Avant l'exécution de ce code, nous faisons l'hypothèse que seule l'inode de la racine (inode numéro 20) est déjà présente dans la table *inode[]*. Donnez la séquence, dans l'ordre, des blocs et des inodes accédés jusqu'à la fin de la ligne 8. Pour chaque accès, spécifiez la ligne de code associée.

Question 5 2 points

Soit la configuration suivante du buffer cache avant l'exécution du code. Le buffer cache contient

4 blocs. Les numéros de bloc sont représentés sous la forme devno-blockno. La fonction de hachage associant un bloc à une des sous-listes est $(\text{devno} + \text{blockno}) \% 5$. Nous considérons que pour tous les blocs $\text{devno} = 0$.



La bfreelist est la suivante (de la tête vers la queue) : 0-201, 0-3, 0-100, 0-4.

Initialement aucun des blocs est marqué avec le flag DELWRI et les writes font des écritures différées.

Donnez la configuration du buffer cache ainsi que la bfreelist et les éventuels flags DELWRI après la ligne 8 du code. Quel est le nombre d'entrées/sorties (E/S) logiques et le nombre d'E/S physiques ? Vous indiquerez sur les quels blocs les E/S ont lieu.

Exercice 3 – Swap (5 points)

Dans cette exercice, on considère que la mémoire comporte 20 unités et la table `proc` ne comporte que 7 processus. Voici l'état de cette table à T_0 juste après l'exécution de la fonction `realtime` qui a conduit à l'élection du processus 103. Dans cette table `p_size` est exprimé en nombre d'unités mémoire :

p_pid	p_uid	p_flag	p_stat	p_size	p_time	p_wchan	p_pri
0	0	SSYS SLOAD	SSLEEP	2	120	&runout	PSWP
1	0	SSYS SLOAD	SSLEEP	4	115	&proc[1]	PWAIT
101	1000	SLOAD	SSLEEP	1	1	&toto	50
102	1000	SLOCK SLOAD	SSLEEP	3	10	&toto	50
103	1000	SLOAD	SRUN	10	0		50
104	1000	0	SSLEEP	10	10	&toto	50
105	1000	0	SSLEEP	10	5	&toto	50

Pour simplifier le scénario, on fait les hypothèses suivantes :

- aucun `fork` ni `exit`, ni `wakeup(&toto)` ne sera appelé par la suite ;
- le temps d'exécution du code noyau (appels systèmes, swapper, init, ...) et les temps d'entrée/sortie sont considérés comme négligeables ;
- on ne prendra pas en compte les segments texte des processus.

Question 1 1 point

On suppose qu'à T_0 le processus 103 exécute l'instruction `kill(104, SIGUSR1)` puis entre dans une boucle infinie d'instructions ne comprenant pas d'appel système.

En vous appuyant sur les codes étudiés en TD, montrez comment l'exécution de cette instruction conduit à réveiller le swapper.

Question 2 2 points

A quelle date le processus 104 va-t-il commencer à s'exécuter pour traiter le signal reçu ?

Pour justifier votre réponse vous indiquerez les actions du swapper depuis T_0 jusqu'à l'exécution du processus 104, en indiquant :

- la date
- le pid du processus concerné
- le sens du transfert : processus swappé ou processus ramené en mémoire

Question 3 1/2 point

Sur quelle adresse le swapper est-il endormi lorsque le processus 104 commence à s'exécuter. Vous justifierez votre réponse.

Question 4 1 1/2 points

En supposant que le signal SIGUSR1 provoque un calcul de 9 secondes dans le processus 104 (le signal n'est pas ignoré et un handler correspondant à ce calcul a été enregistré correctement), combien de temps après l'envoi du signal (à T_0) va se terminer ce calcul ? Justifiez votre réponse.

Exercice 4 – Appel système - Fichiers (7 points)

Question 1 2 points

On souhaite écrire un nouvel appel système `int check_open(int idev, int inumber)`. Cet appel prend en entrée un numéro de partition `idev` et un numéro d'inode `inumber`. La fonction retourne 0, si l'inode de la partition n'est pas en cours d'utilisation par un processus et 1 sinon. Donnez le code dans le noyau de l'appel système `_check_open()` correspondant à `int check_open(int dev, int inumber)`.

Question 2 2 points

On souhaite modifier l'appel système pour retourner 0 si l'inode n'est pas utilisée, 1 si elle est utilisée que par des processus en mode lecture uniquement et 2 si elle est utilisée dans d'autres modes. Donnez le nouveau code de `_check_open()`.

Question 3 3 points

On veut maintenant modifier l'appel système pour que `_check_open()` se bloque à la priorité `PWOPEN` si moins de 3 processus ont précédemment ouvert le fichier en lecture. `_check_open()` doit retourner 0 si le fichier n'est pas utilisé et 1 s'il est utilisé par au moins 3 processus en lecture. Modifiez la fonction `_check_open()` en conséquence et indiquez quelles sont les modifications à appliquer à `iget`, donnée en annexe, pour réveiller le processus.

Annexe

```
#define NADDR 13

struct inode
{
    char          i_flag;
    char          i_count; /* reference count */
    dev_t         i_dev;   /* device where inode resides */
    ino_t         i_number; /* i number, 1-to-1 with device address */
    unsigned short i_mode;
    short         i_nlink; /* directory entries */
    short         i_uid;   /* owner */
    short         i_gid;   /* group of owner */
    off_t         i_size;  /* size of file */
    struct {
        daddr_t    i_addr[NADDR]; /* if normal file/directory */
        daddr_t    i_lastr;        /* last logical block read (for read-
        ahead) */
    };
};

struct inode inode[NINODE]; /* The inode table itself */

/* flags */
#define ILOCK 01 /* inode is locked */
#define IUPD 02 /* file has been modified */
#define IACC 04 /* inode access time to be updated */
#define IMOUNT 010 /* inode is mounted on */
#define IWANT 020 /* some process waiting on lock */
#define ITEXT 040 /* inode is pure text prototype */
#define ICHGO 100 /* inode has been changed */

/* modes */
#define IFMT 0170000 /* type of file */
#define IFDIR 0040000 /* directory */
#define IFCHR 0020000 /* character special */
#define IFBLK 0060000 /* block special */
#define IFREG 0100000 /* regular */
#define IFMPC 0030000 /* multiplexed char special */
#define IFMPB 0070000 /* multiplexed block special */
#define ISUID 04000 /* set user id on execution */
#define ISGID 02000 /* set group id on execution */
#define ISVTX 01000 /* save swapped text even after use */
#define IREAD 0400 /* read, write, execute permissions */
#define IWRITE 0200
#define IEXEC 0100

/*
 * One file structure is allocated
 * for each open/creat/pipe call.
 * Main use is to hold the read/write
 * pointer associated with each open
 * file.
 */
struct file
{
    char          f_flag;
    cnt_t         f_count; /* reference count */
};
```

```
        int      *f_inode;      /* pointer to inode structure */
        long     f_offset;      /* read/write character index */
} file[NFILE];
```

```
/* flags */
```

```
#define FOPEN      (-1)
#define FREAD      0x0001
#define FWRITE     0x0002
#define FPIPE      0x0004
```

```
/* open parameters */
```

```
#define O_RDONLY      0
#define O_WRONLY      1
#define O_RDWR        2
```

```

1  iget (dev,ino)
2  {
3      register struct inode *p;
4      register *ip2;
5      int *ip1;
6      loop:
7          ip = NULL;
8          for (p = &inode[0]; p < &inode[NINODE]; p++){
9              if (dev == p->i_dev && ino == p->i_number ) {
10                 if ((p->i_flag & ILOCK) != 0) {
11                     p->i_flag |= IWANT;
12                     sleep (p,PINOD);
13                     goto loop;
14                 }
15                 p->i_count++;
16                 p->i_flag |= ILOCK;
17                 return (p);
18             }
19             if (ip == NULL && p->i_count == 0)
20                 ip = p;
21         }
22
23         if ((p - ip) == NULL) {
24             printf ("inode_table_overflow\n");
25             u.u_error = ENFILE;
26             return (NULL);
27         }
28         p->i_dev = dev;
29         p->i_number = ino;
30         p->i_flag = ILOCK;
31         p->i_count ++;
32         p->i_lastr --1;
33         ip = bread (dev, ldiv (ino+31,16) );
34
35
36         /* check I/O errors */
37         if (ip->b_flags & B_ERROR) {
38             brelse (ip);
39             iput(p);
40             return (NULL);
41         }
42
43
44         ip1 = ip->b_addr + 32*ldiv(ino+31,16) ;
45         ip2 = &p->i_mode ;
46
47         while (ip2 < &p->i_addr[8])
48             *ip2++ = *ip1++;
49         brelse (ip);
50         return (p);
51 }

```